

Relazione di Progetto: UNO Flip! - Premium Multiplayer Edition

Autori: Donato Umberto Muscillo, Fabio Nigro

Corso: Laboratorio di Informatica

Data di consegna: Giugno 2026

Sommario

1. Analisi
2. Progettazione
3. Codifica
4. Test
5. Conclusioni

1. Analisi

Il progetto consiste nella progettazione e nello sviluppo di una simulazione digitale avanzata del celebre gioco di carte "UNO Flip!", scritta interamente in linguaggio C++17. L'obiettivo cardine risiede nella trasposizione delle complesse meccaniche fisiche del gioco da tavolo — in particolare l'inedita dinamica del mazzo a "doppia faccia" (Lato Chiaro e Lato Oscuro) — all'interno di un software interattivo, responsivo e cross-platform.

Il software supera il concetto di simulazione testuale offrendo un'interfaccia grafica (GUI) bidimensionale premium realizzata tramite la libreria SFML 3. Il sistema è in grado di gestire animazioni fluide, un sistema di collisioni adattivo (hitbox mapping) per l'interazione con il mouse, e un comparto audio generato proceduralmente. L'applicativo supporta il gioco in locale contro un'Intelligenza Artificiale integrata, oppure sfide multigiocatore su rete LAN (Architettura Client-Server) garantendo la sincronizzazione speculare dei mazzi tra sistemi operativi differenti (macOS e Windows) tramite un algoritmo deterministico pseudo-casuale (LCG).

1.1 Funzionalità Principali e Utenti Target

- **Motore di Gioco Completo:** Implementazione fedele del regolamento di UNO Flip!, inclusi effetti speciali come le penalità, il salto del turno e il capovolgimento globale (Flip) delle carte e del colore attivo.
- **Intelligenza Artificiale (Bot):** Sviluppo di agenti autonomi in grado di valutare la validità delle mosse, scegliere i colori ottimali e applicare le regole senza l'intervento umano.
- **Multiplayer Online (Architettura Lockstep):** Possibilità di connettere due dispositivi a

distanza tramite architettura Server-Client su protocollo TCP, garantendo la sincronizzazione assoluta del mazzo e dei turni tra i due nodi di rete.

- **Interfaccia Grafica Premium:** Utilizzo della libreria SFML per renderizzare un menu interattivo con effetti visivi (Glassmorphism, hover dinamici), visualizzazione chiara delle mani dei giocatori e indicatori di turno.
- **Persistenza dei Dati:** Sistema integrato per il salvataggio dei risultati a fine partita e la lettura formattata di una "Hall of Fame" (Classifica Globale) basata su file CSV, con filtro automatico per escludere i Bot dai punteggi umani.

1.2 Requisiti Funzionali

Codice	Nome Funzionalità	Descrizione
R01	Gestione Interfaccia Grafica (GUI)	Renderizzazione del menu principale, bottoni interattivi (con hover effect) e tavoli da gioco tramite la libreria SFML 3.
R02	Inizializzazione Partita e Mazzo	Creazione dinamica del mazzo "doppia faccia" (Chiaro/Oscuro), mescolamento randomico e distribuzione iniziale di 7 carte per giocatore.
R03	Modalità Giocatore Singolo	Possibilità di avviare partite in locale contro avversari controllati dall'Intelligenza Artificiale (Bot).
R04	Logica dell'IA (Bot)	Algoritmo in grado di valutare la mossa migliore (privilegiando carte speciali), selezionare i colori ottimali post-Jolly e pescare se bloccato (approccio Greedy).

Codice	Nome Funzionalità	Descrizione
R05	Multiplayer Architettura di Rete	Creazione di una stanza tramite avvio di un Server TCP (Host) o connessione come Client tramite inserimento di indirizzo IP con socket non-bloccanti.
R06	Sincronizzazione Lockstep	Sincronizzazione in tempo reale del "Seed" randomico tra Host e Client per garantire l'esatta replicazione della generazione e mescolamento del mazzo sui due dispositivi.
R07	Meccaniche di Gioco e Validazione	Controllo di validità sulla carta giocata (corrispondenza di colore, numero o simbolo) rispetto alla carta in cima agli scarti.
R08	Risoluzione Effetti UNO Flip	Applicazione degli effetti speciali: Salta, Inverti, Pesca Carte e la meccanica "FLIP", che capovolge l'intero mazzo e le mani dei giocatori attivando il Lato Oscuro.
R09	Gestione Input Simultanei (Debounce)	Prevenzione di input accidentali multipli (es. doppio click o tap ripetuto) tramite timer di blocco (Debounce) impostato a 0.25s per evitare desincronizzazioni.

Codice	Nome Funzionalità	Descrizione
R10	Meccanica "Contesta UNO"	Sistema asincrono che permette a un giocatore di cliccare un bottone di penalità manuale se l'avversario gioca la penultima carta senza essersi "prenotato" dicendo UNO.
R11	Gestione Disconnessione (Rage Quit)	Intercettazione dell'uscita volontaria di un giocatore dalla rete, con conseguente vittoria a tavolino (Forfait) assegnata al giocatore rimasto.
R12	Lettura e Scrittura Classifica (CSV)	Parsing di un file .csv per l'aggiornamento delle statistiche (Vittorie, Partite Giocate) con esclusioni automatiche dei Bot.

1.3 Casi d'uso (Use Cases)

- **UC1 - Avvio Partita Locale:** L'utente inserisce il proprio nome nel form di Login, accede al Menu Principale e seleziona "Gioca Locale". Il sistema istanzia i giocatori (umano e bot), genera il mazzo e avvia il loop di gioco.
- **UC2 - Ospitare Partita Multiplayer (Host):** L'utente seleziona "Multiplayer" e successivamente "Ospita Partita". Il sistema apre una porta TCP (53000) e si mette in ascolto. Alla ricezione di una connessione in entrata, il sistema scambia i nomi, genera un Seed randomico, lo invia al Client e avvia la partita sincronizzata.
- **UC3 - Partecipare a Partita Multiplayer (Client):** L'utente digita l'indirizzo IP dell'Host e clicca "Unisciti". Il sistema stabilisce la connessione TCP, invia il proprio nome e attende il Seed dal Server per avviare la partita in perfetta sincronia.
- **UC4 - Contestazione Regola "UNO":** Durante il turno avversario, se l'utente nota che

l'avversario è rimasto con una sola carta senza aver dichiarato "UNO", clicca il bottone di contestazione. Il sistema intercetta l'input, invia un pacchetto di rete (se in multiplayer) e assegna istantaneamente 2 carte di penalità al trasgressore.

- **UC5 - Consultazione Classifica:** L'utente clicca "Consulta Classifica". Il sistema legge il file classifica.csv, estrae i record, li ordina in base al numero di vittorie e renderizza la classifica a schermo formattandola in stile "Podio".

2. Progettazione

L'architettura del software si basa sul Pattern MVC (Model-View-Controller): le entità di base come Carta, Mazzo e Giocatore (Model) non hanno alcuna dipendenza grafica e mantengono esclusivamente lo stato logico. La classe Partita (Controller) gestisce le interazioni e le regole matematiche. Le scene grafiche come GameScene (View) gestite dallo SceneManager elaborano l'output visivo.

2.1 Tipi di dato e strutture dati

- **Classi C++ (class):** Utilizzate per modellare entità complesse (Carta, Mazzo, Giocatore, Partita).
- **Enumerazioni (enum):** Colore (Rosso, Verde, Nero, ecc.) e Valore (Zero, Salta, Flip, ecc.) per una gestione typesafe degli attributi delle carte, inclusi gli stati della macchina a stati finiti (StatoGioco).
- **Vettori Dinamici (std::vector):** Utilizzati ampiamente per gestire mazzi e mani dei giocatori, permettendo allocazione dinamica efficiente (push_back, pop_back).
- **Puntatori Smart (std::unique_ptr, std::shared_ptr):** Utilizzati nel SceneManager e nella gestione della Rete per il memory management automatico, evitando memory leak (RAII).

2.2 Librerie e funzioni

Per lo sviluppo sono state utilizzate le seguenti librerie: SFML (Simple and Fast Multimedia Library) per la gestione della finestra, del rendering grafico 2D, degli input da mouse/tastiera/touch e la gestione della rete (moduli Graphics, Window, System, Network); la STL (Standard Template Library) per la gestione dinamica della memoria, manipolazione stringhe e input/output da file (<fstream>).

2.3 Dipendenze tra funzioni

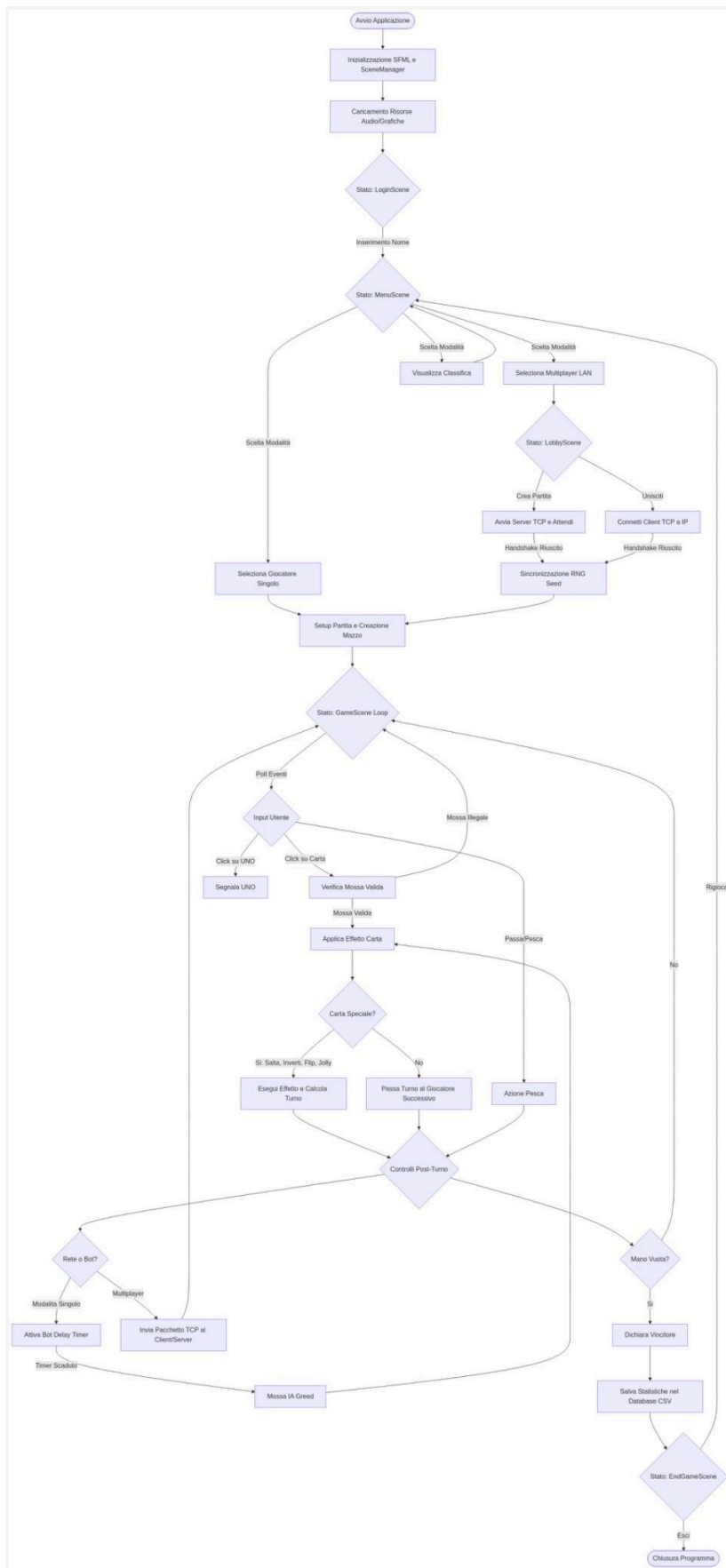
L'architettura del software è basata su una gerarchia di chiamate: Il Main inizializza la finestra e l'oggetto StatoGioco. In base allo stato, vengono richiamate le funzioni di rendering della classe Interfaccia (gestione bottoni e menu). La classe Partita coordina le classi Mazzo e Giocatore. Durante il turno, se il giocatore è remoto, la funzione mossaRete attende i dati dall'oggetto

GestoreRete, creando una dipendenza asincrona tra la logica di gioco e il socket di rete.

2.4 Flow chart dell'Architettura

Per la schematizzazione della logica di sistema, è stato progettato un diagramma di flusso ad albero che descrive il loop asincrono principale e la gestione degli eventi. Di seguito viene riportata la struttura logica del diagramma e il relativo codice sorgente Mermaid per la generazione vettoriale del grafico.

Fase Logica	Descrizione delle Operazioni
1. Setup Iniziale	START APP → LOGIN (Salvataggio Nome) → MENU PRINCIPALE. Dal Menu si dirama la scelta: Classifica (Parser CSV), Locale/Simulazione (Setup Istanza Partita), o Multiplayer.
2. Rete e Sincronia	Selettore LOBBY RETE: Ospita (Apertura TCP) o Unisciti (Connessione IP) → Sincronizzazione SEED Random → Setup Partita.
3. Game Loop	PARTITA IN CORSO → Verifica Eventi Asincroni (Rage Quit, Contestazione) → Controllo Turno (Umano, Bot o Rete).
4. Risoluzione Mossa	Mossa Eseguita (Giocabile o Pesca) → Risoluzione Effetto (Normale, +1/+5, Salta, FLIP) → Check Vittoria (Carte in mano == 0).
5. Termine	Se Carte == 0 → FINE PARTITA → Aggiornamento Database CSV → Ritorno al Menu.



3. Codifica

In questa fase sono stati tradotti in linguaggio C++ i modelli logici definiti durante la progettazione. Di seguito si analizzano i frammenti di codice relativi agli algoritmi più rilevanti e complessi del sistema.

3.1 La Meccanica "FLIP" e la Risoluzione delle Collisioni Cromatiche

Il nucleo del gioco risiede nell'applicazione della carta speciale "FLIP". L'algoritmo deve invertire lo stato globale del mazzo (da Chiaro a Oscuro e viceversa) e aggiornare il colore attivo. Durante i test è emerso un edge-case critico: se il retro della carta giocata risultava essere un "Jolly" (colore Nero), il gioco entrava in stallo poiché nessun utente aveva scelto il colore. L'algoritmo è stato quindi irrobustito forzando una scelta cromatica casuale congruente con il lato appena attivato.

```
else if (vMano == FLIP) {
    latoOscuroAttivo = !latoOscuroAttivo;
    mostraAvviso(latoOscuroAttivo ? "LATO OSCURO ATTIVATO!" : "LATO
    CHIARO ATTIVATO!");
    Colore nuovoColore = c.getColore(latoOscuroAttivo);
    if (nuovoColore == NERO) {
        int r = rand() % 4;
        if (!latoOscuroAttivo) {
            Colore chiari[] = {ROSSO, GIALLO, VERDE, BLU};
            coloreAttivo = chiari[r];
        } else {
            Colore oscuri[] = {ROSA, VERDE_ACQUA, ARANCIONE,
            VIOLA}; coloreAttivo = oscuri[r];
        }
    } else {
        coloreAttivo = nuovoColore;
    }
    passaTurno();
}
```

3.2 Sincronizzazione Rete tramite Deterministic Lockstep

L'uso della funzione standard `std::shuffle` produceva risultati disallineati nel multiplayer cross-platform (Mac vs Windows) a causa delle diverse implementazioni dell'entropia nei

compilatori. Per garantire il Deterministic Lockstep, è stato codificato un Generatore Lineare Congruenziale (LCG) proprietario che, nutrito con il medesimo Seme iniziale generato dall'Host tramite `time(NULL)` ed inviato via handshake TCP, riproduce fedelmente la stessa sequenza di mescolamento su architetture diverse:

```
// LATO SERVER: Genera il seed e lo invia assieme al messaggio di
INIZIO
int randomSeed = time(NULL);
srand(randomSeed);
mioServer.inviaMessaggio("INIZIO|" + nomeInput + "|" +
to_string(randomSeed));
gioco = new Partita(giocatori);
gioco->setupIniziale();

// LATO CLIENT: Riceve il seed, lo imposta e avvia la partita
sincrona
if (dati[0] == "INIZIO") {
    srand(stoi(dati[2])); // Imposta lo stesso seed ricevuto
    vector<Giocatore> giocatori = {Giocatore(dati[1], false),
Giocatore(nomeInput, false)};
    gioco = new Partita(giocatori);
    gioco->setupIniziale();
}
```

3.3 DPI Scaling, Supporto Touch e Debouncing

Nei sistemi Windows moderni, l'ingrandimento predefinito dello schermo (DPI Scaling al 125% o 150%) sfalsava la cattura raw dei pixel, rendendo i pulsanti insensibili ai click. Il codice è stato irrobustito mappando le coordinate dello schermo nello spazio vettoriale globale (`mapPixelToCoords`). Inoltre, è stato implementato un blocco anti-spam (Debounce timer) e il supporto ai tocchi da Trackpad:

```
// Lettura evento click del mouse e touch screen con mappatura
coordinate
sf::Vector2f clickPos;
bool clickRilevato = false;
if (const auto* mouseEv =
event->getIf<sf::Event::MouseButtonPressed>()) {
    if (mouseEv->button == sf::Mouse::Button::Left) {
        clickPos = window.mapPixelToCoords(mouseEv->position);
```

```

        clickRilevato = true;
    }
} else if (const auto* touchEv =
event->getIf<sf::Event::TouchBegan>()) {
    clickPos = window.mapPixelToCoords(touchEv->position);
    clickRilevato = true;
}
if (clickRilevato) {
    if (timerAntiSpam.getElapsedTime().asSeconds() < 0.25f)
continue;
    timerAntiSpam.restart();
    // ... esecuzione della logica del bottone ...
}

```

4. Test

La fase di testing è stata condotta per validare i casi d'uso (UC) e assicurare che ogni requisito funzionale (R) rispondesse in modo coerente.

Codice Requisito	Codice Test	Nome	Descrizione test	Eventuale input	Risultato atteso	Risultato ottenuto
R01	T1.1	Navigazione e Menu	Click sul bottone "Classifica Globale".	Click SX (Mouse)	Transizione allo Stato MENU_CLASSIFICA e rendering del file CSV.	Successo
R07	T1.2	Validazione e Mossa Errata	Tentativo di giocare una carta Rossa su un colore attivo	Click SX su Carta Rossa	La carta non viene scartata, il turno non passa.	Successo

Codice Requisito	Codice Test	Nome	Descrizione test	Eventuale input	Risultato atteso	Risultato ottenuto
			Verde.			
R08	T1.3	Edge-Cas e FLIP cromatico	Giocata carta FLIP il cui retro corrisponde a una carta "Jolly" (Nera).	Giocata carta FLIP	Il gioco non entra in stallo: viene forzato un colore random del nuovo lato attivo.	Successo
R09	T1.4	Spam Input (Debounce)	Doppio/Triplo click ravvicinato su una carta giocabile.	Click multipli (< 0.25s)	Viene elaborato solo il primo click, ignorando i successivi. Nessuna desincronizzazione.	Successo
R10	T1.5	Contestazione UNO	Avversario gioca la penultima carta senza premere il bottone UNO. Il giocatore locale clicca	Click su "CONTESTA UNO!"	L'avversario riceve istantaneamente +2 carte nel suo vettore mano.	Successo

Codice Requisito	Codice Test	Nome	Descrizione test	Eventuale input	Risultato atteso	Risultato ottenuto
			"Contesta" .			
R05/R06	T2.1	Sincronizzazione Rete Lockstep	L'Host avvia il server. Il Client si connette con l'IP.	Input stringa IP e Click "Unisciti"	Il Server accetta, genera il Seed, lo invia. I due client mescolano il mazzo in modo identico.	Successo
R11	T2.2	Intercettazione Rage Quit	Durante una partita multiplayer, il Client clicca "Abbandona".	Click su "ABBANDONA"	Il Server riceve il pacchetto "QUIT", forza la vittoria a tavolino ed espone il pop-up.	Successo
R12	T3.1	Filtro Salvataggio o Bot	Una partita in "Simulazione" viene vinta dal "Bot Luigi".	Vittoria del Bot	La funzione di salvataggio o esclude i bot. Il CSV registra la partita ma ignora la vittoria.	Successo

5. Conclusioni

Il progetto "UNO Flip! - Premium Edition" ha raggiunto con successo tutti gli obiettivi prefissati nella fase di analisi. La trasposizione delle complesse meccaniche del gioco da tavolo fisico in un software C++ ha richiesto una rigorosa architettura orientata agli oggetti (OOP), permettendo di gestire in modo efficiente il cambio di stato globale (Lato Chiaro/Oscuro) e la validazione delle mosse.

Punti di Forza: Architettura di Rete performante tramite Deterministic Lockstep su socket TCP con traffico dati ridotto a pochissimi byte; Interfaccia Utente (UI) premium accelerata via hardware con effetti grafici evoluti, filtro anti-spam e calcolo proporzionale tramite Letterboxing; elevata resilienza del codice nella gestione automatica di edge-case cromatici e strutturali.

Punti di Debolezza e Sviluppi Futuri: Implementazione futura di alberi decisionali evoluti (MinMax o Monte Carlo Tree Search) per dotare i Bot di una visione strategica predittiva a lungo termine; transizione del modulo di networking Peer-to-Peer verso un Master Server con Lobby Rooms gestite globalmente tramite protocolli agili come i WebSockets; centralizzazione cloud della Hall of Fame tramite database relazionali remoti.